

Construcción y evolución del software

Manejo de fallos

Dr. Rodrigo Vázquez López

Universidad Autónoma de la Ciudad de México
Plantel San Lorenzo Tezonco

Semestre 2026-I

UACM

Universidad Autónoma
de la Ciudad de México

NADA HUMANO ME ES AJENO

Contenido

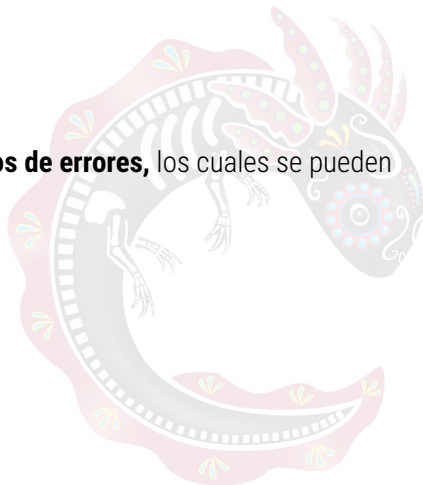
- 1 Errores
 - Errores sintácticos
 - Errores semánticos o lógicos
 - Errores de ejecución
- 2 Excepciones
- 3 Manejo de excepciones
- 4 Excepciones propias



Tipos de errores

Una aplicación puede tener diversos **tipos de errores**, los cuales se pueden clasificar en **tres grandes grupos**:

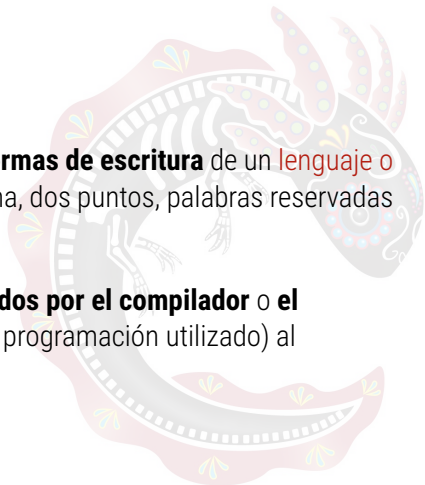
- Errores **sintácticos**.
- Errores **semánticos o lógicos**.
- Errores **de ejecución**.



Errores sintácticos

Son fallas generadas por **infringir las normas de escritura** de un **lenguaje o sintaxis**; por ejemplo, coma, punto y coma, dos puntos, palabras reservadas mal escritas, entre otras.

Normalmente estos errores **son detectados por el compilador o el intérprete** (dependiendo del lenguaje de programación utilizado) al **procesar** el código fuente.



Errores sintácticos

Ejemplos

1 Falta un punto y coma.

```
1 int x = 5  
2 System.out.println(x);
```

2 Variables no declaradas

```
1 System.out.println(myVar);
```

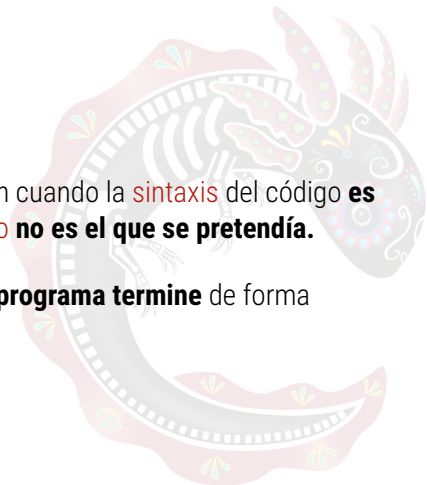
3 Incompatibilidad de tipos

```
1 int x = "Hello";
```

Errores semánticos o lógicos

Tienen **efectos más sutiles**, se producen cuando la **sintaxis** del código **es correcta**, pero la **semántica o significado no es el que se pretendía**.

Un **error semántico** puede hacer que el **programa termine** de forma **anormal** o con **resultados incoherentes**.



Errores semánticos o lógicos

Ejemplo

1 Resultado no esperado

```
1 int x = 10;  
2 int y = 2;  
3 int sum = x - y;  
4 System.out.println("x + y = " + sum);
```

2 Comportamiento inadecuado

```
1 int value;  
2 while(value = 1) {  
3     System.out.println("Active");  
4 }
```

Errores de ejecución

Se presentan cuando la aplicación **se está ejecutando** e **impiden que continúe** con este proceso. Su **origen es diverso**, se pueden producir por un

- **Uso incorrecto** del programa **por parte del usuario**: si ingresa una cadena cuando se espera un número.
- **Errores de programación**: acceder a **localidades no reservadas** o hacer **divisiones entre cero**.
- Algún **recurso externo** al programa: al acceder a un **archivo** o al conectarse a algún **servidor** o cuando se acaba el espacio en la **pila o stack** de la memoria.

Errores de ejecución

Ejemplo:

1 División entre cero:

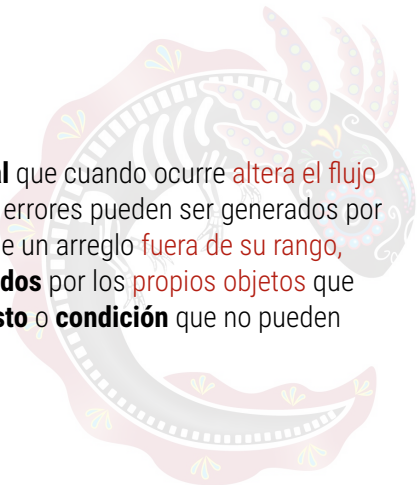
```
1 int x = 10;  
2 int y = 0;  
3 int result = x / y;  
4 System.out.println(result);
```

2 Acceder a un elemento fuera de los límites del arreglo

```
1 int[] numbers = {1, 2, 3};  
2 System.out.println(numbers[8]);
```

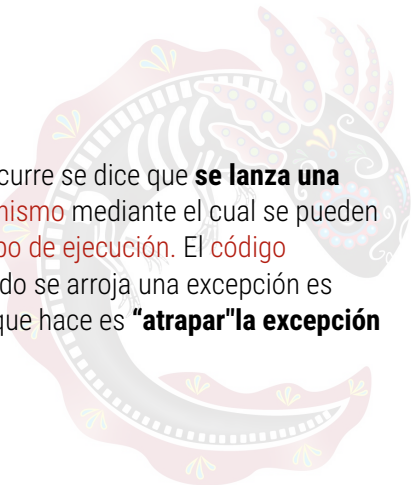
Excepciones

Una **excepción** es una **condición anormal** que cuando ocurre **altera el flujo normal** del programa en ejecución. Estos errores pueden ser generados por la **lógica del programa**, como un **índice** de un arreglo **fuera de su rango**, una **división entre cero** o **errores generados** por los **propios objetos** que denuncian algún tipo de **estado no previsto** o **condición** que no pueden manejar.



Excepciones en Java

En **Java**, cuando un **evento excepcional** ocurre se dice que **se lanza una excepción**. Las excepciones son **el mecanismo** mediante el cual se pueden **controlar los errores** producidos en **tiempo de ejecución**. El **código responsable** de hacer alguna acción cuando se arroja una excepción es llamado **manejador de excepciones** y lo que hace es **"atrapar" la excepción lanzada**.



Manejo de excepciones

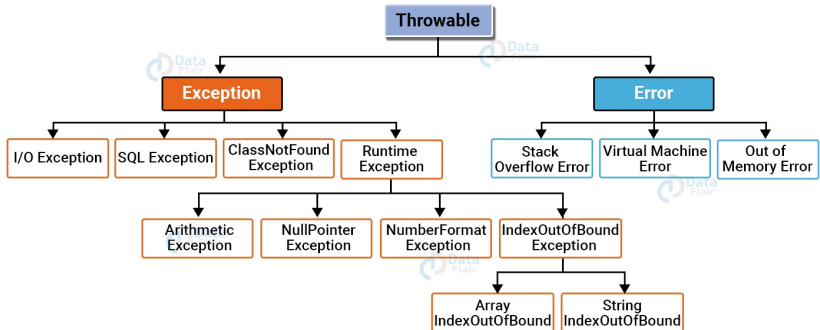
Para **manejar una excepción** se utilizan las palabras reservadas `try` y `catch`. El **bloque try** es utilizado para **definir** el bloque de código en el cual **una excepción puede ocurrir**. El o **los bloques catch** son utilizados para **definir** un bloque de código que **maneje la excepción**.

```
1 try {  
2     // Block of code to try  
3 }  
4 catch(Exception e) {  
5     // Block of code to handle errors  
6 }
```

Las excepciones son **objetos** que **contienen información** del error que se ha producido; heredan de la clase `Exception` que a su vez hereda de la clase `Throwable`.

Jerarquía de las excepciones

Hierarchy of Java Exceptions



Manejo de excepciones

Ejemplo:

```
1 public class Main {  
2     public static void main(String[ ] args) {  
3         int[] myNumbers = {1, 2, 3};  
4         System.out.println(myNumbers[10]); // error!  
5     }  
6 }
```

Si un código **lanza una excepción** y ésta **no se captura**, entonces interviene un **manejador por defecto** que normalmente **imprime información** que ayuda a **encontrar dónde y cómo se produjo el error**. Sin embargo, **como la excepción no fue controlada**, ésta **se propaga hasta llegar al método principal** y **termina abruptamente el programa**.

Manejo de excepciones

Ejemplo:

Utilizando bloques `try-catch`

```
1 public class Main {  
2     public static void main(String[ ] args) {  
3         try {  
4             int[] myNumbers = {1, 2, 3};  
5             System.out.println(myNumbers[10]);  
6         } catch (Exception e) {  
7             System.out.println("Something went wrong.");  
8         }  
9     }  
10 }
```

Bloque finally

La palabra reservada **finally** permite definir un **tercer bloque** de código dentro del **manejador de excepciones**. Este bloque le indica al programa las **instrucciones a ejecutar** de manera **independiente** de los **bloques try-catch**, es decir, si el código del **bloque try** se ejecuta de **manera correcta, entra al bloque finally**; si se **genera un error**, después de ejecutar el código del **bloque catch** ejecuta el código del **bloque finally**.

```
1 try {  
2     // Block of code to try  
3 }  
4 catch(Exception e) {  
5     // Block of code to handle errors  
6 }  
7 finally {  
8     // Block of code to execute always!  
9 }
```


Bloque finally

Ejemplo:

```
1 public class Main {  
2     public static void main(String[] args) {  
3         try {  
4             int[] myNumbers = {1, 2, 3};  
5             System.out.println(myNumbers[10]);  
6         } catch (Exception e) {  
7             System.out.println("Something went wrong.");  
8         } finally {  
9             System.out.println("The 'try catch' is finished.");  
10        };  
11    }  
12 }
```

Algunas excepciones importantes

Error/Excepción	Descripción
<code>ArithmeticException</code>	Ocurre cuando una operación o cálculo numérico produce un error.
<code>ArrayIndexOutOfBoundsException</code>	Ocurre cuando se intenta acceder a una posición de un arreglo que no existe .
<code>ClassNotFoundException</code>	Ocurre cuando se intenta acceder a una clase que no existe o no puede localizarse .
<code>FileNotFoundException</code>	Ocurre cuando no es posible acceder a un archivo especificado.
<code>InputMismatchException</code>	Ocurre cuando se introduce un tipo de dato incorrecto en la entrada (por ejemplo, texto en lugar de un valor numérico).
<code>IOException</code>	Ocurre cuando falla una operación de entrada o salida .
<code>NullPointerException</code>	Ocurre cuando se intenta acceder a una referencia de objeto cuyo valor es <code>null</code> .
<code>NumberFormatException</code>	Ocurre cuando no es posible convertir una cadena de texto al tipo numérico especificado.
<code>StringIndexOutOfBoundsException</code>	Ocurre cuando se intenta acceder a un carácter de una cadena en una posición que no existe .

Múltiples bloques catch

Es posible tener más de un bloque Bloque `catch` dentro del **manejador de excepciones**:

```
1 try {  
2     // Block of code to try  
3 }  
4 catch(ArithmeticException e) {  
5     // Block of code to handle arithmetic errors  
6 }  
7 catch(ClassNotFoundException e) {  
8     // Block of code to handle class error  
9 }  
10 catch(Exception e) {  
11     // Block of code to handle general errors  
12 }  
13 finally {  
14     // Block of code to execute always!  
15 }
```

Múltiples bloques catch

Ejemplo

```
1 public class Main {  
2     public static void main(String[] args) {  
3         int[] numbers = {0, 1, 2, 3};  
4         for(int i=0; i<=numbers.length; i++) {  
5             float res = 10/numbers[i];  
6             System.out.println("Res= " + res);  
7         }  
8         System.out.println("Program has ended!");  
9     }  
10 }
```

Múltiples bloques catch

Ejemplo:

```
1 public class Main {  
2     public static void main(String[] args) {  
3         int[] numbers = {0, 1, 2, 3};  
4         try{  
5             for(int i=0; i<=numbers.length; i++) {  
6                 float res = 10/numbers[i];  
7                 System.out.println("Res= " + res);  
8             }  
9         } catch (ArithmeticException e) {  
10            System.out.println("Arithmetic error");  
11        } catch (ArrayIndexOutOfBoundsException e) {  
12            System.out.println("Index error");  
13        } finally {  
14            System.out.println("The rock has come back");  
15        }  
16        System.out.println("Program has ended!");  
17    }  
18 }
```

Consideraciones sobre el manejo de múltiples excepciones

Se debe cuidar el **orden de captura** de las excepciones acomodándolas de las **más específicas** a las **más generales**.

Si se **invierte el orden**, se capturan las excepciones **de lo más general a lo más específico**, **todas las excepciones** caerían en las primeras (`Exception` es la más general) y **no habría manera** de enviar un error del tipo `IOException` o `ClassNotFoundException`.

Multi-catch

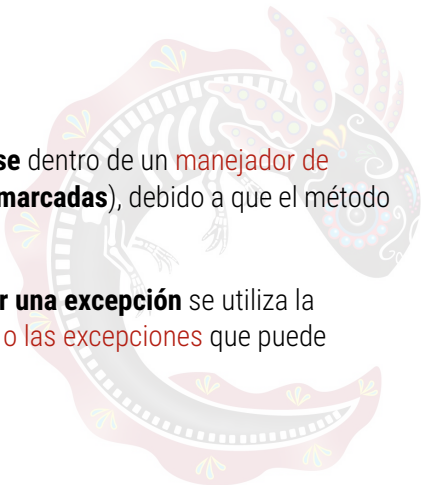
Desde Java 7, puedes capturar **múltiples excepciones** en un bloque **catch** usando el símbolo **|**. Esto es útil cuando **diferentes excepciones** deben manejarse de la **misma manera** para no repetir código.

```
1 public class Main {  
2     public static void main(String[] args) {  
3         try {  
4             int result = 10 / 0;  
5             int[] numbers = {1, 2, 3};  
6             System.out.println(numbers[10]);  
7         }  
8         catch (ArithmeticException |  
9               ArrayIndexOutOfBoundsException e) {  
10             System.out.println("Math error or array error  
11             occurred.");  
12         }  
13     }  
14 }
```

Instrucción throws

Existen métodos que **obligan a ejecutarse** dentro de un **manejador de excepciones** (es decir, son **excepciones marcadas**), debido a que el método define que va a **lanzar una excepción**.

Para indicar que un método **puede lanzar una excepción** se utiliza la palabra reservada **throws** seguida de **la o las excepciones** que puede arrojar dicho método.



Instrucción throws

Ejemplo

```
1 public static int myMethod(int a, int b) throws  
    ArithmeticException {  
2     int c = a / b;  
3     return c;  
4 }
```

Esta sintaxis **obliga** que el método que mande llamar a `myMethod()` debe **manejar la excepción** o **indicar** que va a arrojar la misma excepción (propagar).

Instrucción throws

```
1 public class Main {
2     public static int metodo(int a, int b) throws
        ArithmeticException {
3         int c = a/b;
4         return c;
5     }
6
7     public static void main(String[] args) {
8         try{
9             int res = metodo(10, 0);
10            System.out.println(res);
11        } catch(Exception e) {
12            System.out.println("Arithmetic exception");
13            e.printStackTrace();
14        }
15        System.out.println("Program has ended!");
16    }
17 }
```

Excepciones propias

La instrucción `throw` permite crear un **error personalizado**.

```
1 public class Main {  
2     public static boolean verify(int age) throws  
        ArithmeticException {  
3         if(age <18) {  
4             throw new ArithmeticException("Access denied!");  
5         }  
6         return true;  
7     }  
8  
9     public static void main(String[] args) throws  
        ArithmeticException {  
10        verify(18);  
11    }  
12 }
```

Excepciones propias

En este caso, la instrucción `throw` permitió generar un **error personalizado** utilizando como base la excepción `ArithmeticException`. De la misma forma, podemos utilizar como **base cualquier excepción** existente.

Sin embargo, cuando se **desarrollan aplicaciones** es común requerir excepciones que **no están definidas** en el API de Java, es decir, **excepciones propias** del **negocio**.

Excepciones propias

Es posible **crear clases** que se comporten como **excepciones** (que se puedan arrojar), para ello sólo es necesario **heredar** de la clase `Exception`. Por ejemplo, podemos escribir **nuestra propia clase** `AgeException`

```
1 public class AgeException extends Exception {  
2     public AgeException() {  
3         super("Access Denied!");  
4     }  
5 }
```

Excepciones propias

```
1 public class Main {
2     public static boolean verify(int age) throws
        AgeException {
3         if(age <18) {
4             throw new AgeException();
5         }
6         return true;
7     }

8
9     public static void main(String[] args) {
10        try {
11            verify(10);
12        } catch (AgeException e) {
13            e.printStackTrace();
14        }
15        System.out.println("Program has ended!");
16    }
17 }
```

Bibliografía

- [1] Francisco Blasco. *Programación orientada a objetos en Java*. Ra-Ma Editorial, 2019.
- [2] Michael Kölling y David J Barnes. *Programación orientada a objetos con Java*. PEARSON EDUCACIÓN, SA, 2007.
- [3] Pablo Sznajdleder. *Java a fondo: curso de programación*. Alpha Editorial, 2016.

